

Finite Element Solver for Rotordynamics

User Manual

Contents

1	Introduction and Software Structure	3
2	Rotor Data Structure	3
2.1	<code>Rotor.nodes</code> — Mesh Nodes	4
2.2	<code>Rotor.shaft</code> — Beam Elements	4
2.3	<code>Rotor.disk</code> — Rigid Disks	6
2.4	<code>Rotor.bearing</code> — Supports and Bearings	6
2.5	<code>Rotor.forcing</code> — Excitations	11
3	Mesh Generation: <code>meshGenerator</code>	12
4	System Matrices Nomenclature	13
5	Solution Scheme — Linear Analysis (Frequency)	14
5.1	Main Function: <code>charRoots</code>	14
5.2	Critical Speeds: <code>critSpeeds</code>	15
5.3	Forced Frequency Response: <code>FRF</code>	15
6	Solution Scheme — Nonlinear Analysis (Time Domain)	15
6.1	Main Function: <code>timeSimulation</code>	15
6.2	Run-up: <code>runUp</code>	16
6.3	Nonlinear Bearing Forces: <code>nonLinBearingMatrix</code>	16
6.4	Modal Reduction: <code>reductionMatrix</code>	16
6.5	ODE Solver Tolerances	16
7	Post-processing Functions	17

8	Utility Functions	17
9	Complete Example — Linear Analysis	17
10	Complete Example — Nonlinear Analysis	19

1 Introduction and Software Structure

The software implements a finite element solver for the rotordynamic analysis of rotating shafts. It supports modal analyses (natural frequencies, Campbell diagrams, critical speeds), forced frequency response analysis, and time-domain simulations with nonlinear behaviors.

The solver is organized into three functional levels:

1. **Model Definition:** construction of the Rotor data structure that describes geometry, materials, and boundary conditions.
2. **Computation:** matrix assembly functions and solution of the eigenvalue problem or ODE.
3. **Post-processing:** functions for visualization and results analysis.

2 Rotor Data Structure

All the information describing the rotordynamic system is contained in a single MATLAB structure named `Rotor` (or `rotor` in the conventions adopted in the example scripts). The mandatory fields are:

Field	Content
<code>Rotor.nodes</code>	Axial coordinates of the mesh nodes
<code>Rotor.shaft</code>	Definition of the shaft beam elements
<code>Rotor.disk</code>	Characteristics of the rigid disks
<code>Rotor.bearing</code>	Definition of supports and bearings
<code>Rotor.forcing</code>	Type of excitation (unbalance, gravity, bow)

Optional fields used by some types of forcing:

<code>Rotor.bend</code>	Definition of plastic deformation (shaft bow)
-------------------------	---

The degrees of freedom (DOFs) for each node follow the order

$$[u_i, v_i, \theta_i, \psi_i]$$

where u and v are the lateral displacements in the x and y directions, and θ, ψ are the associated rotations. The total number of DOFs is $n_{\text{dof}} = 4 \times n_{\text{nodes}}$.

2.1 `Rotor.nodes` — Mesh Nodes

Each node is defined by a progressive index and its axial coordinate.

Field	Type	Description
<code>.node</code>	int	Node index (from 1 to n_{nodes})
<code>.coord</code>	double	Axial coordinate z of the node [m]

Listing 1: Example of node definition

```

1 numEl = 8;
2 L_shaft = 1.6;    % [m]
3 L_El = L_shaft / numEl;
4
5 nodes    = num2cell(1:numEl+1);
6 z_nodes  = num2cell(0:L_El:L_shaft);
7 Rotor.nodes = struct('node', nodes, 'coord', z_nodes);

```

Alternatively, the `meshGenerator` function automatically builds the `Rotor.nodes` and `Rotor.shaft` structures starting from a coarse geometric description with optional refinement (see Section 3).

2.2 `Rotor.shaft` — Beam Elements

Each beam element connects two consecutive nodes and is characterized by a formulation, cross-sectional geometric properties, and material properties.

Beam Element Types

Type	Formulation	Shear	Rot. Inertia	Gyroscopic
1	Euler-Bernoulli	—	—	✓
2	Timoshenko (Cowper 1966)	✓	✓	✓
3	Timoshenko (Hutchinson 2001)	✓	✓	✓
21	Tapered, Euler-Bernoulli	—	—	✓
22	Tapered, Timoshenko (Cowper)	✓	✓	✓
23	Tapered, Timoshenko (Hutchinson)	✓	✓	✓

Table 1: Available beam element types.

Notes:

- Type 3 (Hutchinson) provides a more accurate shear coefficient for tubular sections compared to type 2 (Cowper).
- Tapered elements (types 21–23) do not support stiffness correction for torque effects.
- The internal damping coefficient β is defined as a ratio to the stiffness matrix: $\mathbf{C}_{\text{int}} = \beta \mathbf{K}$.
- For tapered elements, the diameters must be specified at both nodes (`d_ext2`, `d_int2`).

Fields of the `Rotor.shaft` Structure

Field	Type	Description
<code>.type</code>	int	Element type (1, 2, 3, 21, 22, 23)
<code>.node1</code>	int	Start node of the element
<code>.node2</code>	int	End node of the element
<code>.d_ext1</code>	double	Outer diameter at node 1 [m]
<code>.d_int1</code>	double	Inner diameter at node 1 [m] (0 for solid section)
<code>.d_ext2</code>	double	Outer diameter at node 2 [m] (only types 21–23)
<code>.d_int2</code>	double	Inner diameter at node 2 [m] (only types 21–23)
<code>.rho</code>	double	Material density [kg/m ³]
<code>.E</code>	double	Young's modulus [Pa]
<code>.G</code>	double	Shear modulus [Pa]
<code>.beta</code>	double	Internal damping coefficient [s] (<i>opt.</i>)
<code>.AxialForce</code>	double	Axial force on the element [N] (<i>opt.</i>)
<code>.Torque</code>	double	Torque on the element [N·m] (<i>opt.</i>)

Table 2: Fields of the `Rotor.shaft` structure.**Listing 2: Example of shaft elements definition (Timoshenko, solid section)**

```

1 E = 200e9;    % [Pa]
2 G = E / 2 / (1 + 0.285);
3 rho = 7800;   % [kg/m^3]
4
5 [Rotor.shaft.type] = deal(2);    % Timoshenko Cowper
6 [Rotor.shaft.rho]  = deal(rho);
7 [Rotor.shaft.E]    = deal(E);

```

```

8 [Rotor.shaft.G]      = deal(G);
9 [Rotor.shaft.d_int1] = deal(0);      % solid section
10 [Rotor.shaft.d_int2] = deal(0);
11 [Rotor.shaft.d_ext1] = deal(0.075); % d = 75 mm
12 [Rotor.shaft.d_ext2] = deal(0.075);
13 node1_cell = num2cell(1:numEl);
14 node2_cell = num2cell(2:numEl+1);
15 [Rotor.shaft.node1] = node1_cell{:};
16 [Rotor.shaft.node2] = node2_cell{:};

```

2.3 Rotor.disk — Rigid Disks

Type	Definition	Required Fields
1	Geometric	<code>.node</code> , <code>.D_ext</code> [m], <code>.D_int</code> [m], <code>.thick</code> [m], <code>.rho</code> [kg/m³]
2	Inertial properties	<code>.node</code> , <code>.mass</code> [kg], <code>.Ip</code> [kg·m²], <code>.Id</code> [kg·m²]

Table 3: Available disk types.

Unbalance parameters (optional, added to the disk structure):

<code>.m0</code>	Unbalance mass m_0 [kg]; equivalence $m_0 \cdot a = m\epsilon$ where $a = D_{\text{ext}}/2$
<code>.epsilon</code>	Mass eccentricity ϵ [m]; alternative to <code>m0</code>
<code>.delta</code>	Unbalance phase angle [rad], measured counterclock- wise from x
<code>.beta</code>	Disk slope angle (slope unbalance) [rad]
<code>.gamma</code>	Slope phase angle [rad]

Listing 3: Example of disk definition (type 2 with unbalance)

```

1 Rotor.disk(1) = struct('type', 2, 'node', 5, ...
2                       'mass', 2.3, 'Id', 3.5e-6, 'Ip', 4e-6);
3 Rotor.disk(1).epsilon = 1e-6; % eccentricity [m]
4 Rotor.disk(1).delta   = 0;    % phase [rad]

```

2.4 Rotor.bearing — Supports and Bearings

The model implements 13 types of supports and bearings. For each bearing, the `.node` field (application node index) and `.type` field (type number) are always required. Additional fields depend on the type.

Type 1 — Pinned Constraint

Eliminates the translation DOFs at the node, allowing rotations. It does not introduce stiffness or damping.

Constrained DOFs: u_i, v_i

Field	Value
<code>.type</code>	1
<code>.node</code>	node index

Type 2 — Clamped Constraint

Eliminates all four DOFs at the node (translations and rotations).

Constrained DOFs: $u_i, v_i, \theta_i, \psi_i$

Field	Value
<code>.type</code>	2
<code>.node</code>	node index

Type 3 — Diagonal linear bearing, translations only

Introduces uncoupled diagonal stiffness and damping in the x and y directions. Rotations are unconstrained.

Field	Unit	Description
<code>.kx</code>	N/m	Stiffness in x direction
<code>.ky</code>	N/m	Stiffness in y direction
<code>.cx</code>	N·s/m	Damping in x direction
<code>.cy</code>	N·s/m	Damping in y direction

Type 4 — Diagonal linear bearing, all DOFs

Extension of type 3 with stiffness and damping for all 4 node DOFs (translations and rotations).

Field	Type	Description
<code>.k</code>	vector 4×1 [N/m, N·m/rad]	$[k_x, k_y, k_\theta, k_\psi]^T$
<code>.c</code>	vector 4×1 [N·s/m, ...]	$[c_x, c_y, c_\theta, c_\psi]^T$

Type 5 — Linear bearing, 2×2 matrices, translations only

Allows cross-coupling between the x and y directions for translations. Rotations are unconstrained.

Field	Type	Description
.K	matrix 2×2 [N/m]	$\begin{bmatrix} k_{xx} & k_{xy} \\ k_{yx} & k_{yy} \end{bmatrix}$
.C	matrix 2×2 [N·s/m]	$\begin{bmatrix} c_{xx} & c_{xy} \\ c_{yx} & c_{yy} \end{bmatrix}$

Type 6 — Full linear bearing, 4×4 matrices

Most general model: full stiffness and damping matrices for all node DOFs.

Field	Type	Description
.K	matrix 4×4	Full stiffness matrix [N/m, N·m/rad, mixed]
.C	matrix 4×4	Full damping matrix

Type 7 — Oil film journal bearing (linear model, short bearing)

Linearized model based on Ocvirk's theory (short bearing, $L/D \ll 1$). Stiffness and damping matrices are automatically computed from the equilibrium eccentricity ratio, starting from static load conditions. The rotational speed must be non-zero.

Field	Unit	Description
.F	N	Static load on the bearing
.D	m	Bearing diameter
.L	m	Bearing axial length
.c	m	Radial clearance
.eta	N·s/m ²	Dynamic oil viscosity
.Linear	0/1	If 1: suppresses K and C (eccentricity only) (<i>opt.</i>)

Type 8 — Pressurized fluid seal

Seal model for centrifugal pumps and liquid pressurized fluid applications (Childs, bulk-flow model). Introduces non-symmetric stiffness and damping on translations.

Field	Unit	Description
.P	Pa	Pressure difference across the seal
.R	m	Seal radius
.L	m	Seal axial length
.c	m	Radial clearance
.V	m/s	Average axial flow velocity
.fric	—	Average friction coefficient

Type 9 — Ball bearing (simplified model)

Kramer's model (1993) for ball bearings. Introduces only diagonal terms on translations; does not account for out-of-plane effects. Valid for 8, 12, or 16 balls.

Field	Unit	Description
.n_ball	—	Number of balls: 8, 12, or 16
.d_ball	m	Ball diameter
.fs	N	Static load on the bearing
.alpha	rad	Contact angle

Type 10 — Squeeze Film Damper (with/without centering spring)

Adams' model (2001) for oil film dampers. Assumes small L/D (short bearing approximation). Introduces only diagonal terms on translations.

Field	Unit	Description
.eta	N·s/m ²	Dynamic oil viscosity
.R	m	Damper radius
.L	m	Damper axial length
.c	m	Radial clearance
.ks	N/m	Centering spring stiffness (<i>opt., default 0</i>)

Type 11 — Thomas-Alford forces (steam/gas turbines)

Model for the destabilizing Thomas-Alford effect in high-pressure turbine stages. Introduces an antisymmetric cross-coupled stiffness with no associated damping.

Field	Unit	Description
.beta	—	Empirical factor (typically $2 < \beta < 5$)
.T	N·m	Torque at the turbine stage
.D	m	Mean blade diameter
.L	m	Radial blade length

The generated cross-coupled stiffness is $K_{sw} = \beta T / (DL)$, with matrix:

$$\mathbf{K}_{TA} = \begin{bmatrix} 0 & K_{sw} \\ -K_{sw} & 0 \end{bmatrix}$$

Type 12 — Labyrinth seal (Scharer-Childs model)

Bulk-flow model for labyrinth seals for compressible gases (Childs & Scharer, 1986). Computes the pressure distribution among the teeth (choked or unchoked), the tangential swirl velocity for each cavity, and the rotordynamic coefficients K , k , C , c via the auxiliary function `labySeals`. Seal parameters must be defined in the bearing; the function is called internally by `bearingMatrix`.

Field	Unit	Description
<code>.sealType</code>	string	'TOS' (teeth on stator) or 'TOR' (teeth on rotor)
<code>.P_reserv</code>	Pa	Upstream pressure (reservoir)
<code>.P_sump</code>	Pa	Downstream pressure (sump)
<code>.Nt</code>	—	Number of teeth
<code>.Cr</code>	m	Radial clearance at the tooth
<code>.L</code>	m	Axial pitch between teeth
<code>.T</code>	K	Gas temperature
<code>.Rs</code>	m	Rotor radius inside the seal
<code>.B</code>	m	Cavity depth
<code>.U_inlet</code>	m/s	Inlet tangential velocity (swirl)
<code>.omega</code>	rad/s	Rotor angular velocity
<code>.nu</code>	m ² /s	Kinematic viscosity of the gas

Type 13 — Speed-dependent stiffness and damping bearing

The K and C matrices are linearly interpolated from tables as a function of the rotational speed. Useful for oil film bearings with coefficients derived from TEHD (Thermo-Elasto-Hydrodynamic) codes or experimental measurements.

Field	Unit	Description
<code>.Speed_points</code>	rad/s	Table speed vector
<code>.kxx_points</code>	N/m	k_{xx} at table points
<code>.kxy_points</code>	N/m	k_{xy} at table points
<code>.kyx_points</code>	N/m	k_{yx} at table points
<code>.kyy_points</code>	N/m	k_{yy} at table points
<code>.cxx_points</code>	N·s/m	c_{xx} at table points
<code>.cxy_points</code>	N·s/m	c_{xy} at table points
<code>.cyx_points</code>	N·s/m	c_{yx} at table points
<code>.cyy_points</code>	N·s/m	c_{yy} at table points

Summary Table — All Bearing Types

Type	Model	Additional Fields
1	Pinned constraint	—
2	Clamped constraint	—
3	Diag. linear, transl. only	<code>kx</code> , <code>ky</code> , <code>cx</code> , <code>cy</code>
4	Diag. linear, all DOFs	<code>k</code> (4×1), <code>c</code> (4×1)
5	Linear, 2×2 matrix	<code>K</code> (2×2), <code>C</code> (2×2)
6	Linear, 4×4 matrix	<code>K</code> (4×4), <code>C</code> (4×4)
7	Oil film (linear)	<code>F</code> , <code>D</code> , <code>L</code> , <code>c</code> , <code>eta</code>
8	Pressurized fluid seal	<code>P</code> , <code>R</code> , <code>L</code> , <code>c</code> , <code>V</code> , <code>fric</code>
9	Ball bearing	<code>n_ball</code> , <code>d_ball</code> , <code>fs</code> , <code>alpha</code>
10	Squeeze Film Damper	<code>eta</code> , <code>R</code> , <code>L</code> , <code>c</code> , <code>ks</code>
11	Thomas-Alford forces	<code>beta</code> , <code>T</code> , <code>D</code> , <code>L</code>
12	Labyrinth seal (gas)	<code>sealType</code> , <code>P_reserv</code> , <code>P_sump</code> , <code>Nt</code> , <code>Cr</code> , <code>L</code> , <code>T</code> , <code>Rs</code> , <code>B</code> , <code>U_inlet</code> , <code>omega</code> , <code>nu</code>
13	K , C tabulated vs speed	<code>Speed_points</code> , <code>kxx/kxy/kyx/kyy_points</code> , <code>cxx/cxy/cyx/cyy_points</code>

Table 4: Summary of available bearing/support types.

2.5 `Rotor.forcing` — Excitations

Specifies the type of excitation applied to the rotor. Multiple types can be combined simultaneously.

Type	Excitation	Description
1	Mass unbalance	Centrifugal force due to eccentric mass or mass eccentricity ε . Parameters in the disk: <code>m0</code> or <code>epsilon</code> , <code>delta</code> (phase). Equivalence: $m_0 a = m \varepsilon$ with $a = D_{\text{ext}}/2$.
2	Slope unbalance	Moment due to the slope of the principal disk axis with respect to the rotation axis. Parameters in the disk: <code>beta</code> (angle), <code>gamma</code> (phase).
3	Shaft bow (plastic deformation)	Equivalent force generated by a permanent deviation of the shaft from the neutral line. Deviation points are defined in the auxiliary structure <code>Rotor.bend</code> : fields <code>.node</code> , <code>.x</code> , <code>.y</code> [m]. The deviation at the other nodes is calculated by Guyan condensation.
4	Gravity	Gravity force applied in the $-y$ direction to all nodes ($g = 9.80665 \text{ m/s}^2$). Necessary for nonlinear simulations with oil film bearings.

Table 5: Available excitation types.

Listing 4: Example of forcing definition

```

1 % Mass unbalance + gravity
2 Rotor.forcing(1).type = 1;
3 Rotor.forcing(2).type = 4;
4
5 % Shaft bow unbalance
6 Rotor.forcing(1).type = 3;
7 Rotor.bend(1).node = 5;
8 Rotor.bend(1).x = 0.2e-3;    % deviation in x [m]
9 Rotor.bend(1).y = 0.0;
```

3 Mesh Generation: `meshGenerator`

The `meshGenerator` function allows building the `Rotor.nodes` and `Rotor.shaft` structures starting from a coarse geometric description, with the possibility to refine each element with intermediate nodes and to handle tapered sections.

Listing 5: Function signature

```

1 Rotor = meshGenerator(z_coords, d_shaftEl, n_intNodes, isGraded,
...

```

```
2      shaftElType, shaftElProperties, shaftLoad,
      intDamping)
```

Parameter	Dimensions	Description
z_coords	$(N + 1) \times 1$	Axial coordinates of the coarse mesh [m]
d_shaftEl	$N \times 4$	Diameters $[d_{i1}, d_{e1}, d_{i2}, d_{e2}]$ for each element [m]
n_intNodes	$N \times 1$	Number of intermediate nodes per element
isGraded	$N \times 1$	1: cosine distribution; 0: uniform
shaftElType	$N \times 1$	Element type (1, 2, 3, 21, 22, 23)
shaftElProperties	$N \times 3$	$[\rho, E, \nu]$ per element
shaftLoad	$N \times 2$	$[F_{ass}, T]$ per element (<i>opt.</i>)
intDamping	$N \times 1$	Coefficient β per element (<i>opt.</i>)

Note: `meshGenerator` does not populate the `Rotor.disk`, `Rotor.bearing`, and `Rotor.forcing` fields, which must be added manually after the call.

4 System Matrices Nomenclature

The equation of motion of the rotordynamic system is:

$$\mathbf{M}\ddot{\mathbf{x}} + (\mathbf{C}_0 + \mathbf{C}_b + \Omega \mathbf{C}_1) \dot{\mathbf{x}} + (\mathbf{K}_0 + \mathbf{K}_b + \Omega \mathbf{K}_1) \mathbf{x} = \mathbf{F}(t, \Omega)$$

Symbol	Meaning
$\mathbf{M} = \mathbf{M}_0 + \mathbf{M}_b$	Global mass matrix (shaft + bearings)
$\mathbf{M}_0$	Shaft mass matrix (consistent, from <code>rotorMatrix</code>)
$\mathbf{M}_b$	Bearing mass matrix (from <code>bearingMatrix</code>)
$\mathbf{C}_0$	Structural damping matrix: $\mathbf{C}_0 = \sum_e \beta_e \mathbf{K}_{0,e}$
$\mathbf{C}_1$	Gyroscopic matrix: term $\Omega \mathbf{C}_1$ in the EOM
$\mathbf{C}_b$	Bearing damping matrix
$\mathbf{K}_0$	Shaft elastic stiffness matrix
$\mathbf{K}_1$	Speed-dependent stiffness matrix (internal damping): term $\Omega \mathbf{K}_1$
$\mathbf{K}_b$	Bearing stiffness matrix

5 Solution Scheme — Linear Analysis (Frequency)

5.1 Main Function: `charRoots`

Computes natural frequencies, mode shapes, and whirl ratios of the entire rotordynamic system as a function of the rotational speed.

1. Matrix assembly — `rotorMatrix`, `bearingMatrix`

- `shaftElement` or `shaftTaperEl`: element matrices for each shaft section (mass, stiffness, gyroscopic, internal damping)
- `diskElement`: mass and gyroscopic matrix for each disk
- `bearingMatrix`: matrices $\mathbf{M}_b$, $\mathbf{C}_b$, $\mathbf{K}_b$ and constrained DOFs

2. State-space formulation — The state matrix $\mathbf{AA}(\Omega)$ is:

$$\mathbf{AA}(\Omega) = \begin{bmatrix} \mathbf{0} & \mathbf{I} \\ -\mathbf{M}^{-1}\mathbf{K} & -\mathbf{M}^{-1}\mathbf{C} \end{bmatrix}$$

where $\mathbf{K} = \mathbf{K}_0 + \mathbf{K}_b + \Omega\mathbf{K}_1$ and $\mathbf{C} = \mathbf{C}_0 + \mathbf{C}_b + \Omega\mathbf{C}_1$.

- #### 3. Eigenvalue/eigenvector computation — Via `eig(AA)`. The complex eigenvalues $\lambda = \sigma + i\omega_d$ contain the growth rate σ [1/s] and the damped frequency ω_d [rad/s]. Overdamped modes ($|\text{Im}(\lambda)| < 10^{-1}$ rad/s) are filtered out.
- #### 4. Whirl ratio computation — `modeWhirl`: the parameter $\kappa \in [-1, 1]$ is computed for each mode, with $\kappa > 0$ for forward whirl (FW) and $\kappa < 0$ for backward whirl (BW).

5.2 Critical Speeds: `critSpeeds`

Method	Description
1	Direct: solves the eigenvalue problem with $\Omega = \Omega_c$ such that the excitation frequency $N_X \Omega_c$ coincides with the natural frequency. Unique and direct solution.
2	Iterative (without initial guess): starts from a nominal speed and converges by successive updating of the system matrix.
3	Iterative (with initial guesses): same as method 2 but with user-provided initial guesses for each critical speed. Recommended when critical speeds are closely spaced.

5.3 Forced Frequency Response: `FRF`

Computes the steady-state response to the complex force vector (unbalance + shaft bow) by solving the linear system at each speed point:

$$\left(-\Omega^2 \mathbf{M} + i\Omega \mathbf{C} + \mathbf{K}\right) \mathbf{X} = \mathbf{F}(\Omega)$$

where $\mathbf{F} = \mathbf{F}_{\text{bow}} + \Omega^2 \mathbf{F}_{\text{unb}}$.

6 Solution Scheme — Nonlinear Analysis (Time Domain)

6.1 Main Function: `timeSimulation`

Numerically integrates the nonlinear equations of motion at constant speed $\Omega = \text{const}$. Uses the stiff solver `ode15s` (Adams-BDF). The solution includes the transient and steady-state regimes; the steady state is extracted from the tail of the time history.

Listing 6: Function signature

```
1 [time, q, qdot] = timeSimulation(Rotor, t_span, rotorSpeed, ...
2                               initPos, initVel, nr=0)
```

- `t_span` = `[t_start, t_end]`: integration interval [s]
- `rotorSpeed`: constant rotational speed [rad/s]

- `initPos`, `initVel`: initial condition vectors of size `ndof`
- `nr` (optional): number of modes for modal reduction; 0 = full model

6.2 Run-up: `runUp`

Version of `timeSimulation` for time-varying speed according to the quadratic polynomial law:

$$\phi(t) = \alpha_1 t^2 + \alpha_2 t + \alpha_3 \quad \Rightarrow \quad \Omega(t) = 2\alpha_1 t + \alpha_2$$

Listing 7: Function signature

```
1 [time, q, qdot, speed] = runUp(Rotor, t_span, alpha, ...
2                               initPos, initVel, nr=0)
```

where `alpha` = [`alpha_1`, `alpha_2`, `alpha_3`].

6.3 Nonlinear Bearing Forces: `nonLinBearingMatrix`

Calculated internally at each time step by the ODE integrator. Implements Adiletta's model (1995) for short oil film bearings (type 7.1) according to Ocvirk's π -film theory. Required fields in the bearing: `D`, `L`, `c`, `eta`.

6.4 Modal Reduction: `reductionMatrix`

Simplified Craig-Bampton reduction option, based on undamped and non-gyroscopic modes at the reference speed. Reduces the system to `nr` modal coordinates. Recommended for:

- Long time simulations with large models
- Run-up analyses where the computational cost is high

The bandwidth of the reduced model is estimated by the function as the natural frequency of the highest retained mode (field `maxPuls` [Hz]).

6.5 ODE Solver Tolerances

Function	RelTol	AbsTol
<code>timeSimulation</code>	10^{-3}	10^{-6}
<code>runUp</code>	10^{-4}	10^{-6}

7 Post-processing Functions

Function	Description
<code>figureRotor</code>	2D schematic of the rotor (shaft, disks, bearings)
<code>plotCampbell</code>	Campbell diagram with NX lines and FW/BW coding
<code>plotRootLocus</code>	Root locus and stability map vs speed
<code>plotLogDecrement</code>	Logarithmic decrement vs speed
<code>plotFRF</code>	Frequency response — Bode (amplitude + phase)
<code>plotMode</code>	3D non-dimensional mode shape with nodal orbits
<code>plotOrbit</code>	Precession orbits in the (u, v) plane
<code>plotDisplacement</code>	Lateral deformation profile along the axis
<code>plotStressAnalysis</code>	3D map of the Von Mises safety factor
<code>plotBifurcation</code>	Bifurcation diagram (Poincaré section vs speed)
<code>poincareMap</code>	Poincaré map at fixed speed
<code>waterfallPlot</code>	Waterfall diagram (spectrum vs speed)
<code>DFT</code>	Fourier transform with spline resampling

Table 6: Visualization and post-processing functions.

8 Utility Functions

Function	Description
<code>meshGenerator</code>	Automatic mesh generation with refinement
<code>massRotor</code>	Calculation of total rotor mass
<code>LogDecrement</code>	Calculation of logarithmic decrement from eigenvalues
<code>sortModesMAC</code>	Mode sorting via MAC (Modal Assurance Criterion)
<code>computeDisplacement</code>	Displacement interpolation with shape functions
<code>stressAnalysis</code>	Von Mises strength analysis (static safety factor)
<code>viscosityTemp</code>	Oil viscosity vs temperature (ASTM D 341)
<code>labySeals</code>	Calculation of labyrinth seal rotordynamic coefficients

Table 7: Utility functions.

9 Complete Example — Linear Analysis

Listing 8: Typical structure of a complete linear analysis

```

1  %% 1. Mesh generation
2  Rotor = meshGenerator(z_coords, d_shaftEl, n_intNodes, ...
3                        isGraded, shaftElType, shaftElProperties);
4
5  %% 2. Disks
6  Rotor.disk(1) = struct('type', 2, 'node', 5, ...
7                        'mass', 2.3, 'Id', 3.5e-6, 'Ip', 4e-6);
8  Rotor.disk(1).epsilon = 1e-6;
9
10 %% 3. Bearings
11 Rotor.bearing(1) = struct('type', 1, 'node', 1);      % pinned
12 Rotor.bearing(2) = struct('type', 7, 'node', 15, ... % oil film
13                          'F', 500, 'D', 0.1, 'L', 0.03, 'c', 2e-4, 'eta', 0.03);
14
15 %% 4. Forcing
16 Rotor.forcing(1).type = 1;      % mass unbalance
17
18 %% 5. Natural frequencies calculation and Campbell diagram
19 RotorSpeed = (0:100:5000) * pi/30; % [rad/s]
20 [evals, evecs, kappa] = charRoots(Rotor, RotorSpeed);
21 [evals_s, evecs_s, kappa_s] = sortModesMAC(evals, evecs, kappa);
22
23 NX = [1 2]; isDamped = true;
24 plotCampbell(RotorSpeed, evals_s(1:6,:), NX, isDamped, kappa_s
25             (:,1:6,:));
26
27 %% 6. Critical speeds
28 [omega_c, modes_c] = critSpeeds(Rotor, 1, isDamped, 2, "num_crit
29                               ", 4);
30 fprintf('1st crit. speed: %.1f rpm\n', omega_c(1)*30/pi);
31
32 %% 7. Unbalance response
33 response = FRF(Rotor, RotorSpeed);
34 plotFRF(RotorSpeed*30/pi, response, [5.1, 5.2]); % node 5, x
35                                     and y dir
36
37 %% 8. Stability
38 plotRootLocus(RotorSpeed*30/pi, evals_s(1,:));
39 plotLogDecrement(RotorSpeed*30/pi, evals_s(1:6,:));

```

10 Complete Example — Nonlinear Analysis

Listing 9: Typical structure of a nonlinear analysis (constant speed)

```

1  %% (Rotor definition as above, with type 7.1 bearings)
2
3  ndof = 4 * numel(Rotor.nodes);
4  Omega = 1500 * pi/30;    % 1500 rpm [rad/s]
5
6  initPos = zeros(ndof, 1);
7  initVel = zeros(ndof, 1);
8  initPos(2:4:end) = 1e-5; % small initial perturbation in y
9
10 %% Time simulation (with modal reduction to 8 modes)
11 [t, q, qd] = timeSimulation(Rotor, [0, 10], Omega, ...
12                               initPos, initVel, nr=8);
13
14 %% Steady-state extraction (last 20% of time)
15 idx_ss = t > 0.8 * t(end);
16 node = 5;
17 x_ss = q(idx_ss, 4*node-3);
18 y_ss = q(idx_ss, 4*node-2);
19
20 %% Visualization
21 poincareMap(t(idx_ss), x_ss, y_ss, Omega);
22
23 %% Run-up 0 -> 3000 rpm in 30 s
24 alpha_low = [(3000*pi/30) / (2*30), 0, 0];
25 [t_up, q_up, ~, spd] = runUp(Rotor, [0,30], alpha_low, ...
26                               zeros(ndof,1), zeros(ndof,1), nr
27                               =8);
28 disp_ampl = sqrt(q_up(:,4*node-3).^2 + q_up(:,4*node-2).^2);
29 figure; plot(spd*30/pi, disp_ampl);
30 xlabel('Speed [rpm]'); ylabel('Amplitude [m]');
31 title('Run-up response');

```